

Trickski Angle Classifier

By Calvin DeBellis



Problem and Motivation

- Collegiate Water Skier for Madison
 - Trickskiing: Smooth ski with no fins
 - Spins and tricks which are paired together for point values
 - 20 Second timeframe to combine unique tricks to score the most points!
 - Tournaments runs are usually visually graded by judges who watch live
 - Scoring system is very old, outdated and prone to error!
- IDEA: Automatic Trick Skiing judge!



Approach and Methodology

- Break down the problem:
- 20 second run video
- break it down into individual tricks (eg 360 etc)
- Break down 1-2 second clip of a trick into frames of a Trick skier
- Classify Orientation to then be able to piece together what trick is performed!
- GOAL: Given image of trick skier. Classify weather they are facing, forward, left, right, or back!



Orientation: Left

Data and Experiments (Data)

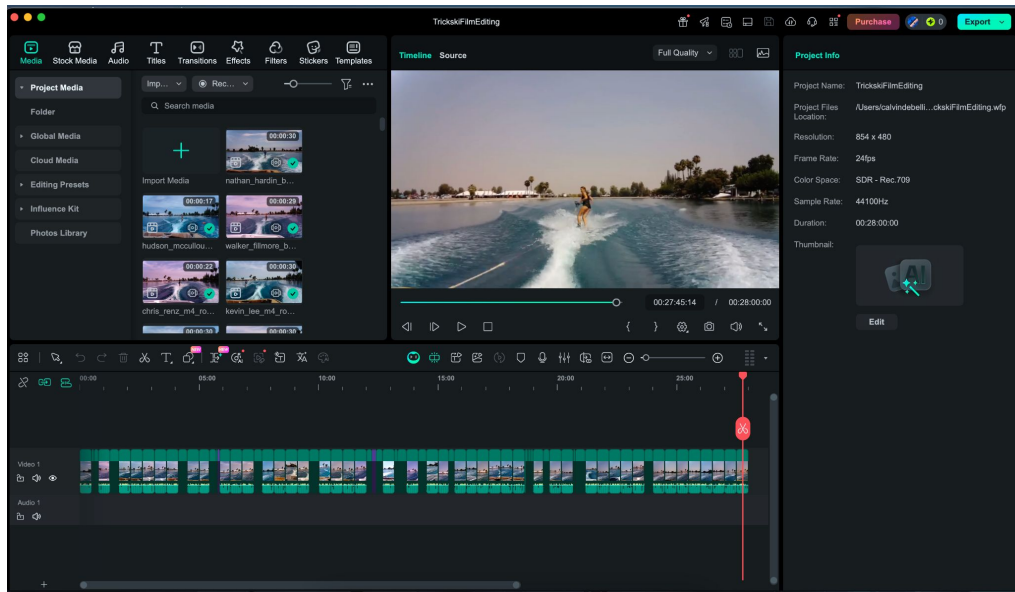
- Curated data myself

- Very Niche topic and there was no known datasets to train on

- Reached out to “waterski results” website manager. Got navigated to database where you can download videos of tournament runs.

- Watched movies and manually downloaded, edited, labeled around 60 videos

- Ended with about 240 photos total for dataset with orientations: “forward”, “left”, “right”, “back”



Data and Experiments (Experiments)

-Started with small in house CNN which performed insanely poorly at around ~ 6% testing accuracy! (may have had data preprocessing errors)

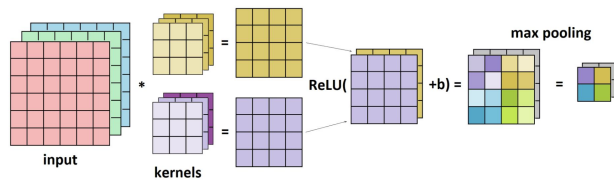
-Finetuned hyperparameters but only got the testing accuracy up to about 10% tops which showed my model wasn't really learning anything meaningful given the loss was steadily decreasing!

-Attempted data augmentation, but only got to 20% testing accuracy. Though the augmentation added a bit more data.

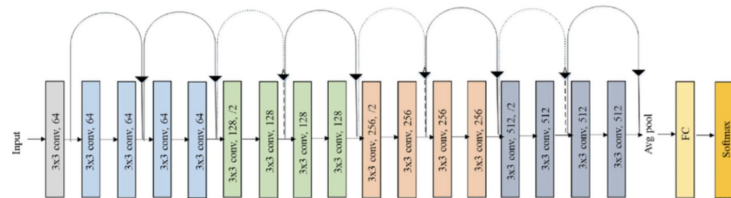
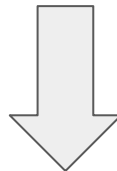
-Switched to pretrained model! (Resnet-18 on ImageNet). Instantly I was able to get the testing accuracy up to about 50%! Realized the importance of using a pretrained model as my dataset was tiny and manually took forever to make! So couldn't increase the size easily

-2 HUGE breakthroughs. I realized that I was making a huge mistake trying to augment data as I was sometimes flipping images and also rotating them slightly which was actually doing more harm than helping! Also realized that there was a lot of noise in the screen due to background and you only really needed to look at the skier!!!

-Fixed the augmentation getting rid of it all together and sticking with the just the pretrained model. Added image cropping to the center to focus on the skier and I was able to get the testing accuracy up to around 80-85% which is where I am currently at!



Simple CNN



Pretrained ResNet-18
on ImageNet



Results and Findings

- As I just said I was able to achieve 80-85% testing accuracy!
- Found that using minimal data it was imperative that I leveraged a pretrained model like ResNet-18 on ImageNet!
- Importance of avoiding any unnecessary noise at all costs! (Use of cropping)
- CAUTION: using data augmentation cause it has the possibility of causing way more harm than good!
- In the end I would say my approach worked pretty well. 80-85% is satisfactory and probably good enough to move on to potential next phases with this project. I would love to expand and make the cropping more dynamic so the accuracy can be even more precise!

Insights and Future Work

- Learned a ton about dataset curation and the difficulties of limited data
- Learned how to use pretrained models and caution with data augmentation
- Learned seemingly difficult AI problems can be broken down into smaller simpler tasks
- So much potential for improvement!:
- Dynamic cropping of skier by using the water ski rope to locate the skier.
- Next step is to take a set of frames from a video, classify orientation of each frame, then piece together the orientation pattern into a trick! Then assign and keep track of point values for each trick.
- End goal would be to upload a 20 second video and be able to classify and score a whole trickski run from start to finish!



